

EC2 and S3 Implementation Using Terraform

Main.tf:

Step 1: Provider Block (Connecting Terraform to AWS)

```
GNU nano 7.2 main.tf
# Provider
provider "aws" {
  region = "us-east-1"
}
```

(Without this block, Terraform doesn't know **which cloud** to talk to).

Step 2: Data Source – Fetch Latest Amazon Linux AMI

```
# Data Source (Get latest Amazon Linux AMI)
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
}
```

(This does NOT create anything. It only fetches information from AWS. Here, we are asking: “AWS, give me the latest Amazon Linux AMI”).

Step 3: Filter Block for AMI Selection

```
filter {
  name   = "name"
  values = ["amzn2-ami-hvm-*-x86_64-gp2"]
}
}
```

(Terraform searches AWS and picks the newest matching AMI, This avoids hard-coding AMI IDs).

Step 4: EC2 Instance Creation

```
# EC2 Instance
resource "aws_instance" "ec2_server" {
  ami          = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"

  tags = {
    Name = "Terraform-EC2"
  }
}
```

(Using the AMI fetched from the data source: Terraform creates an EC2 instance).

Step 5: S3 Bucket Resource (Storage Creation)

```
# S3 Bucket (Secure)
resource "aws_s3_bucket" "secure_bucket" {
  bucket = "my-secure-terraform-bucket-arugnata"
}
```

(Terraform creates an Amazon S3 bucket to store data. Name is globally unique & follows AWS naming rules (lowercase and hyphens only).

Step 6: Blocking Public Access to S3 (Security)

```
# Block public access
resource "aws_s3_bucket_public_access_block" "block_public" {
  bucket = aws_s3_bucket.secure_bucket.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}
```

(Public access is completely blocked. Even accidental public permissions are denied).

❖ Terraform State Management

```
aaru@Dell:~/terraform-assignment$ ls -a
.  ..  .terraform  .terraform.lock.hcl  main.tf  terraform.tfstate  terraform.tfstate.backup
aaru@Dell:~/terraform-assignment$
```

This file stores:

- Resource IDs
- Resource configurations
- Current infrastructure state

Why State Management is Important?

Terraform uses the state file to:

- Compare desired infrastructure (main.tf)
- With existing infrastructure (AWS)

Based on this comparison, Terraform decides whether to:

- Create new resources
- Modify existing resources
- Delete unused resources

❖ Validation and Resource Creation

After writing the Terraform code, the following commands were executed:

terraform init

terraform validate

terraform plan

terraform apply

Proof of Resource Creation

The screenshot displays the AWS Management Console interface for EC2 instances. At the top, there's a header for 'Instances (1/1)' with buttons for 'Connect', 'Instance state', 'Actions', and 'Launch'. A search bar is present with the text 'Find Instance by attribute or tag (case-sensitive)'. Below the search bar, a filter 'Instance state = running' is applied. The main table lists the instance 'Terraform-EC2' with ID 'i-0ff1857efe84fb18e', state 'Running', type 't2.micro', and '2/2 checks passed'. A red box highlights the instance name and ID. Below the table, the details for 'i-0ff1857efe84fb18e (Terraform-EC2)' are shown. A red box highlights the 'AMI ID' field, which contains 'ami-03f9680ef0c07a3d1'. Other details include 'AMI name' 'amzn2-ami-hvm-2.0.20251208.0-x86_64-gp2', 'Monitoring' 'disabled', 'Allowed image' '-', 'Platform details' 'Linux/UNIX', and 'Termination protection' 'Disabled'.

Name	Instance ID	Instance state	Instance type	Status check
Terraform-EC2	i-0ff1857efe84fb18e	Running	t2.micro	2/2 checks passed

i-0ff1857efe84fb18e (Terraform-EC2)

AMI ID ami-03f9680ef0c07a3d1	Monitoring disabled	Platform details Linux/UNIX
AMI name amzn2-ami-hvm-2.0.20251208.0-x86_64-gp2	Allowed image -	Termination protection Disabled

Fig: EC2 Instance was successfully created using Terraform code.

The screenshot displays the AWS Management Console interface for S3 buckets. At the top, there's a header for 'General purpose buckets (3)' with buttons for 'Copy ARN', 'Empty', and 'Delete'. A search bar is present with the text 'Find buckets by name'. Below the search bar, a table lists the buckets. A red box highlights the bucket 'my-secure-terraform-bucket-arugnata'.

Name	AWS Region	Creation date
arugnat-workshop-s3-bucket	US East (N. Virginia) us-east-1	December 24, 2025, (UTC+05:45)
aryu-devops-terraform	US East (N. Virginia) us-east-1	December 24, 2025, (UTC+05:45)
my-secure-terraform-bucket-arugnata	US East (N. Virginia) us-east-1	December 29, 2025, (UTC+05:45)

Fig: S3 Bucket was successfully created and secured using Terraform code.

Summary of Terraform Workflow

- Terraform reads main.tf
- Downloads the AWS provider plugin (if not already present)
- Fetches the latest AMI information
- Creates an S3 bucket
- Secures the S3 bucket
- Launches an EC2 instance
- Stores infrastructure details in terraform.tfstate